

Concept reflection

- 1. You dropped at least one column as a data-leakage feature. Name the column and explain, in two sentences, why keeping it would have produced a great-looking but useless model.*
 - We dropped “interest_rate” because it’s a leakage feature. Lending Club actually sets the interest rate after deciding the loan grade, so the rate basically gives away the answer. If we kept it, the model would look almost perfect, but it wouldn’t be useful in real life since that information isn’t available when making predictions.
- 2. Your neural network did **not** dramatically beat logistic regression on this dataset. What does that tell you about when neural networks are worth the added complexity?*
 - Neural networks are most useful when the relationship between inputs and outputs is really complex and non-linear, especially with large datasets. In this case, the loan data is structured and the relationships are more straightforward, so logistic regression already does a good job. The neural network adds complexity but doesn’t improve performance much, so it’s not really worth it here.
- 3. If the cost of funding a bad loan is roughly **10 times** the cost of turning away a good borrower, how should the classification threshold shift, and why?*
 - The threshold should be lowered, and we chose 0.10. The default 0.50 assumes both types of mistakes are equally costly, but that’s not true here. Missing a risky borrower (false negative) is much more expensive than incorrectly flagging a safe one.
 - Because of that, we want the model to catch more risky loans, even if it means more false alarms. When we lowered the threshold to 0.10, the model flagged a lot more risky borrowers, which reduced the total cost under the 10:1 cost assumption. Even though precision goes down, the trade-off is worth it because it helps prevent more expensive mistakes.
- 4. Why did your pipeline scale the features **after** the train/test split, not before? What specifically would have gone wrong if you scaled first?*
 - We scaled the data after splitting into train and test sets so that the model doesn’t accidentally learn from the test data. If we scaled everything before splitting, the model would indirectly “see” information from the test set, which would make the results look better than they actually are. By fitting the scaler only on training data, we make sure our evaluation is fair and realistic.
- 5. If you had ordinal-encoded the Lending Club loan grade (A-G) as 1-7 instead of one-hot, what specific modeling problem could that create for logistic regression - and does it matter for decision trees?*

- If we encoded loan grades A through G as numbers (1–7), logistic regression would assume the difference between each grade is evenly spaced, which isn't true in real life. That could lead to incorrect assumptions in the model. One-hot encoding avoids this by treating each grade as separate, letting the model learn their relationships more accurately.
- For decision trees, this isn't really a problem because they don't rely on numerical spacing. They just split the data based on what works best, so the exact numeric values don't matter as much.